

1785345165790_Data_Visualization_Matplotlib

July 29, 2026

#

DATA VISUALIZATION WITH MATPLOTLIB

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and plots, visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data. In Python, Matplotlib is the foundational, most widely-used library for creating static, animated, and interactive visualizations.

Let us dive deeper into Matplotlib, its core structures, and its plotting operations.

0.1 MATPLOTLIB

Matplotlib was first released in 2003 by John D. Hunter. It is a powerful, low-level plotting library built on top of NumPy, and almost every other Python visualization library (including Seaborn and Pandas' `.plot()`) is built on top of it. A Matplotlib visualization fundamentally contains:

1. **Figure:** The entire canvas or window that holds everything, including one or more plots.
2. **Axes:** The individual plot area (with its own x-axis and y-axis) drawn inside the Figure.
3. **Axis:** A single number-line-like object (x-axis or y-axis) that defines graph limits and generates ticks.
4. **Artist:** Everything visible on the Figure — lines, text, legends, and shapes are all “artists”.

0.1.1 Figure

A Figure is the outermost container for a plot — think of it as the blank page or canvas you draw on.

0.1.2 Axes

An Axes is where your actual data gets plotted. A single Figure can contain multiple Axes (subplots), similar to how a photo album can hold multiple photos.

0.1.3 Axis

The Axis objects (x-axis, y-axis) control the scale, limits, and tick marks of a plot.

```
[1]: import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
```

```
print("Matplotlib version:", matplotlib.__version__)
print("NumPy version:", np.__version__)
```

Matplotlib version: 3.10.8

NumPy version: 2.4.4

#

CORE STRUCTURES: FIGURE & AXES

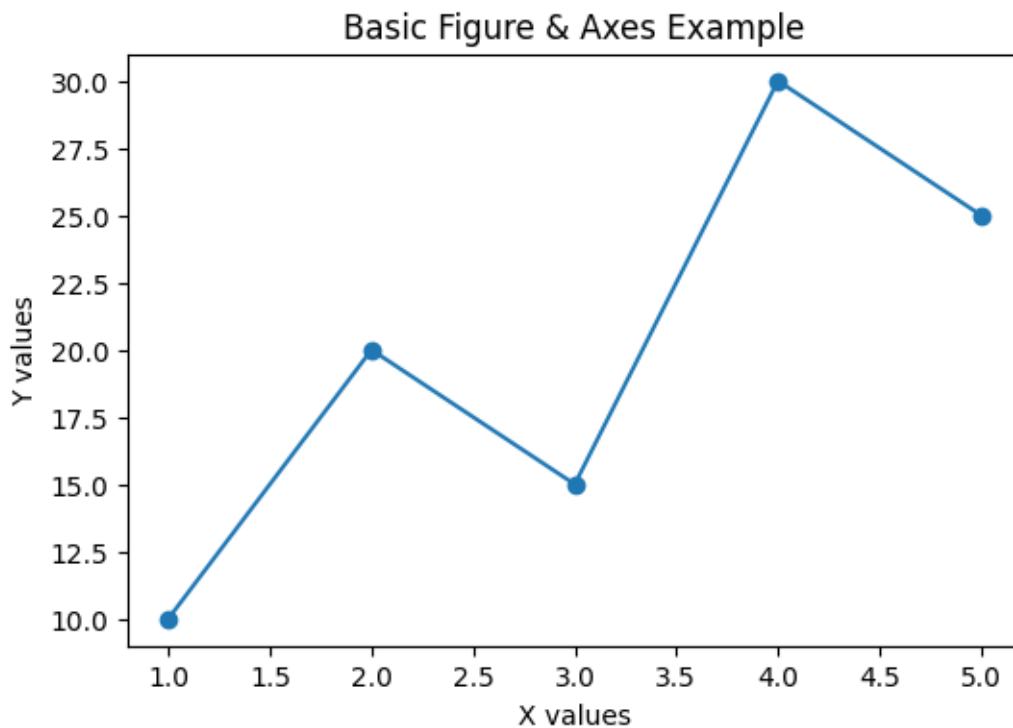
0.1.4 1. Creating a Figure and Axes

Every Matplotlib plot starts with a Figure and at least one Axes object. You can create them explicitly using `plt.subplots()`, which gives full control over the layout. Code snippet to create a basic Figure and Axes:

```
[2]: # Creating a simple Figure and Axes object
fig, ax = plt.subplots(figsize=(6, 4))

ax.plot([1, 2, 3, 4, 5], [10, 20, 15, 30, 25], marker='o', color='#1f77b4')
ax.set_title("Basic Figure & Axes Example")
ax.set_xlabel("X values")
ax.set_ylabel("Y values")

plt.show()
```



0.1.5 2. Multiple Subplots

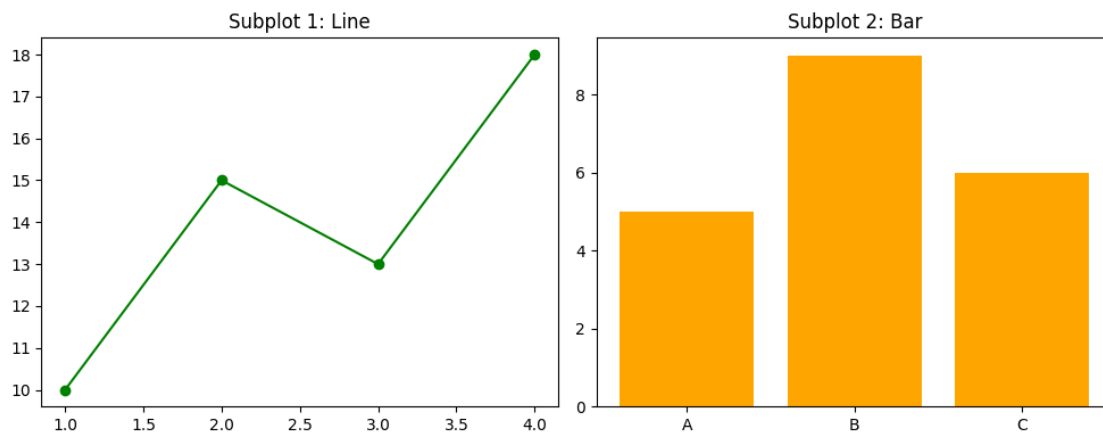
Matplotlib allows multiple Axes inside a single Figure using `plt.subplots(rows, cols)`, which is useful for comparing several charts side-by-side. Code snippet to create a 1x2 subplot grid:

```
[3]: # Creating a Figure with two Axes side-by-side
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].plot([1,2,3,4], [10,15,13,18], color='green', marker='o')
axes[0].set_title("Subplot 1: Line")

axes[1].bar(['A','B','C'], [5, 9, 6], color='orange')
axes[1].set_title("Subplot 2: Bar")

plt.tight_layout()
plt.show()
```



#

PREPARING SAMPLE DATA

Real-world plotting usually starts with a dataset. Let's simulate a business performance table that we'll reuse across several chart types. Code snippet to build the dataset:

```
[4]: # Simulating a dynamic business performance dataset
monthly_metrics = pd.DataFrame({
    'sales_volume': [200, 250, 300, 280, 350, 310, 360, 340, 400, 420, 390, 450],
    'marketing_spend': [30, 35, 40, 38, 50, 45, 55, 52, 60, 65, 58, 70]
}, index=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])

print("--- Monthly Metrics ---")
monthly_metrics
```

--- Monthly Metrics ---

```
[4]:      sales_volume  marketing_spend
     Jan           200             30
     Feb           250             35
     Mar           300             40
     Apr           280             38
     May           350             50
     Jun           310             45
     Jul           360             55
     Aug           340             52
     Sep           400             60
     Oct           420             65
     Nov           390             58
     Dec           450             70
```

#

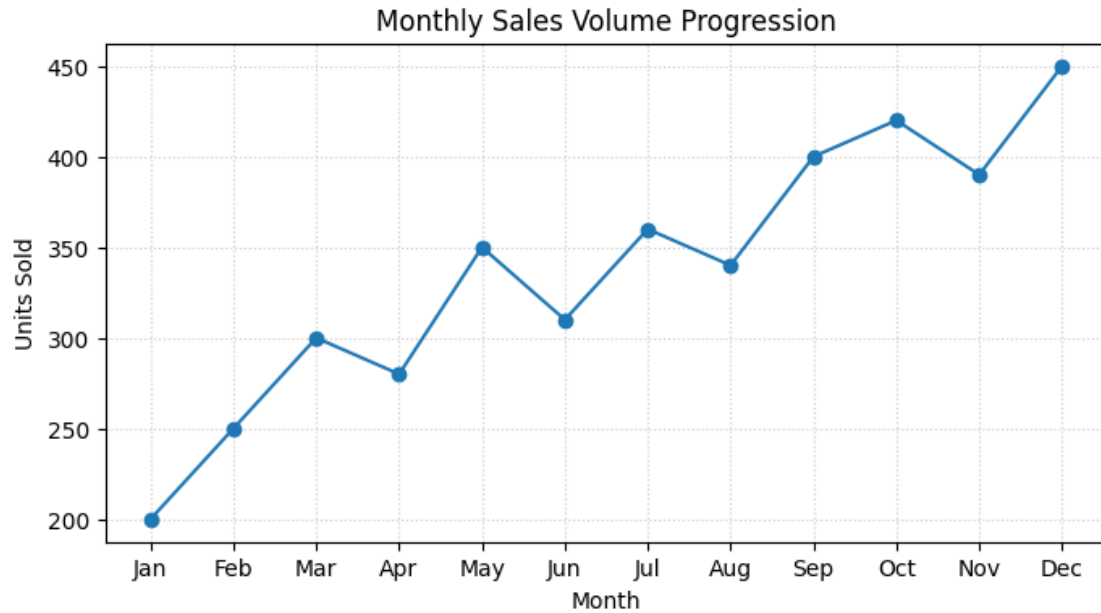
CORE GRAPH TYPES

0.1.6 1. Line Plot

Description: Displays data points connected by a line — ideal for trends over a sequential progression or time index. **Use Case:** Tracking sales, temperature, or stock prices across months. Code snippet:

```
[5]: print("--- Creating a Matplotlib Line Chart ---")
     plt.figure(figsize=(8, 4))
     plt.plot(monthly_metrics.index, monthly_metrics['sales_volume'], marker='o',
     ↪color='#1f77b4')
     plt.title("Monthly Sales Volume Progression")
     plt.xlabel("Month")
     plt.ylabel("Units Sold")
     plt.grid(True, linestyle=':', alpha=0.6)
     plt.show()
```

--- Creating a Matplotlib Line Chart ---

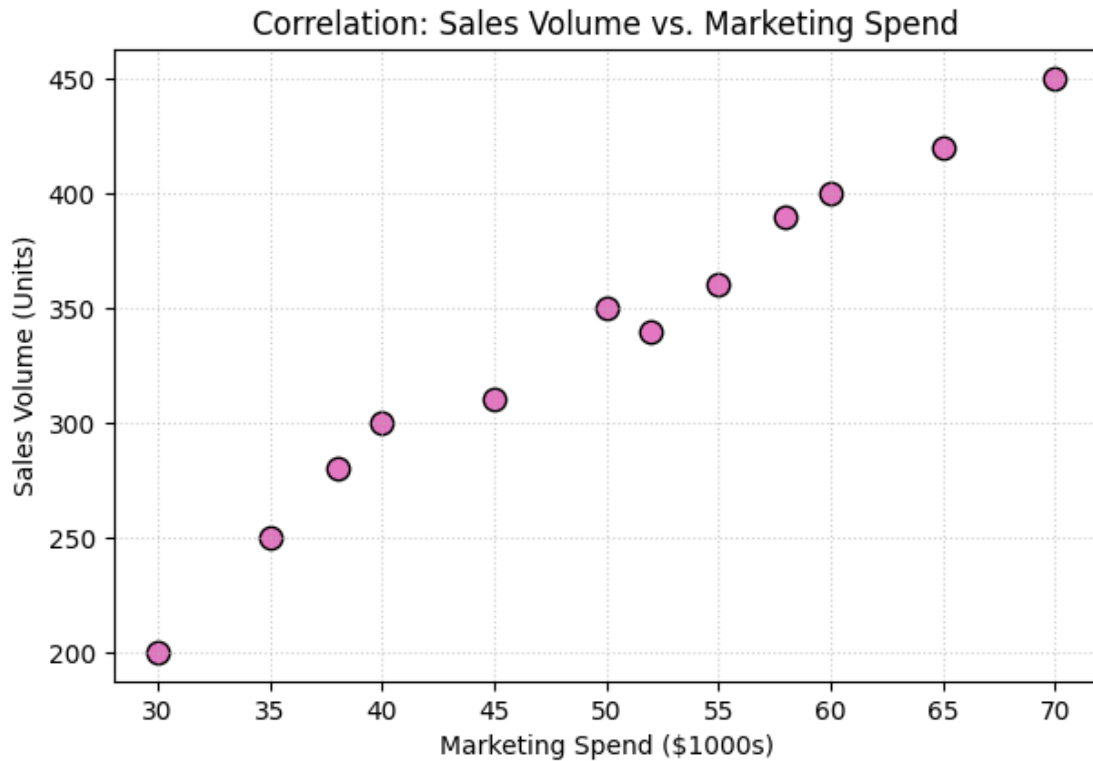


0.1.7 2. Scatter Plot

Description: Displays individual data points on an X-Y plane — useful for revealing relationships between two variables. **Use Case:** Checking whether marketing spend correlates with sales volume. Code snippet:

```
[6]: print("--- Creating a Matplotlib Scatter Plot ---")
plt.figure(figsize=(7, 4.5))
plt.scatter(monthly_metrics['marketing_spend'], monthly_metrics['sales_volume'],
            s=80, color='#e377c2', edgecolor='black')
plt.title("Correlation: Sales Volume vs. Marketing Spend")
plt.xlabel("Marketing Spend ($1000s)")
plt.ylabel("Sales Volume (Units)")
plt.grid(True, linestyle=':', alpha=0.6)
plt.show()
```

--- Creating a Matplotlib Scatter Plot ---

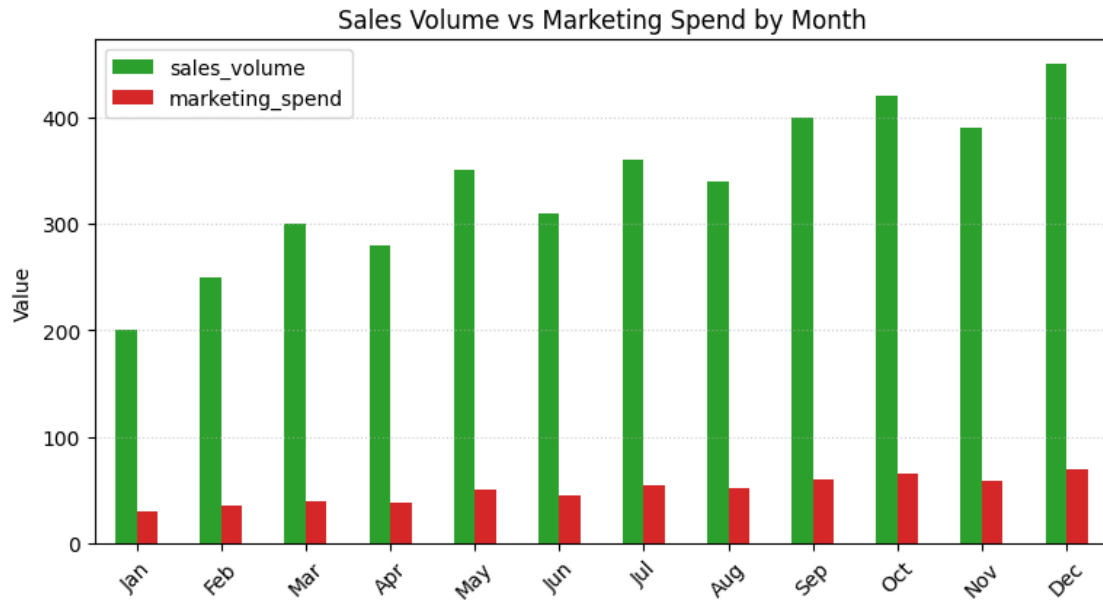


0.1.8 3. Bar Chart

Description: Represents categorical data with rectangular bars — great for comparing quantities side-by-side. **Use Case:** Comparing sales volume vs marketing spend across months. Code snippet:

```
[7]: print("--- Creating a Matplotlib Grouped Bar Chart ---")
monthly_metrics.plot(kind='bar', figsize=(9, 4.5), color=['#2ca02c', '#d62728'])
plt.title("Sales Volume vs Marketing Spend by Month")
plt.ylabel("Value")
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle=':', alpha=0.6)
plt.show()
```

--- Creating a Matplotlib Grouped Bar Chart ---



0.1.9 4. Histogram

Description: Splits a continuous numerical column into frequency “bins” to reveal skewness, multi-modality, or centralized tendencies. **Use Case:** Understanding how purchase order sizes are distributed. Code snippet:

```
[8]: np.random.seed(42)
order_sizes = np.random.normal(loc=150, scale=30, size=500)

print("--- Creating a Matplotlib Histogram ---")
plt.figure(figsize=(8, 4))
plt.hist(order_sizes, bins=25, color='#9467bd', edgecolor='black', alpha=0.8)
plt.title("Distribution of Purchase Order Sizes")
plt.xlabel("Order Value ($)")
plt.ylabel("Frequency")
plt.show()
```

--- Creating a Matplotlib Histogram ---



0.1.10 5. Box Plot

Description: Visually charts the five-number summary of a dataset: minimum, Q1, median, Q3, and maximum, while explicitly highlighting outliers. **Use Case:** Auditing variance and spotting outliers across two retail stores. Code snippet:

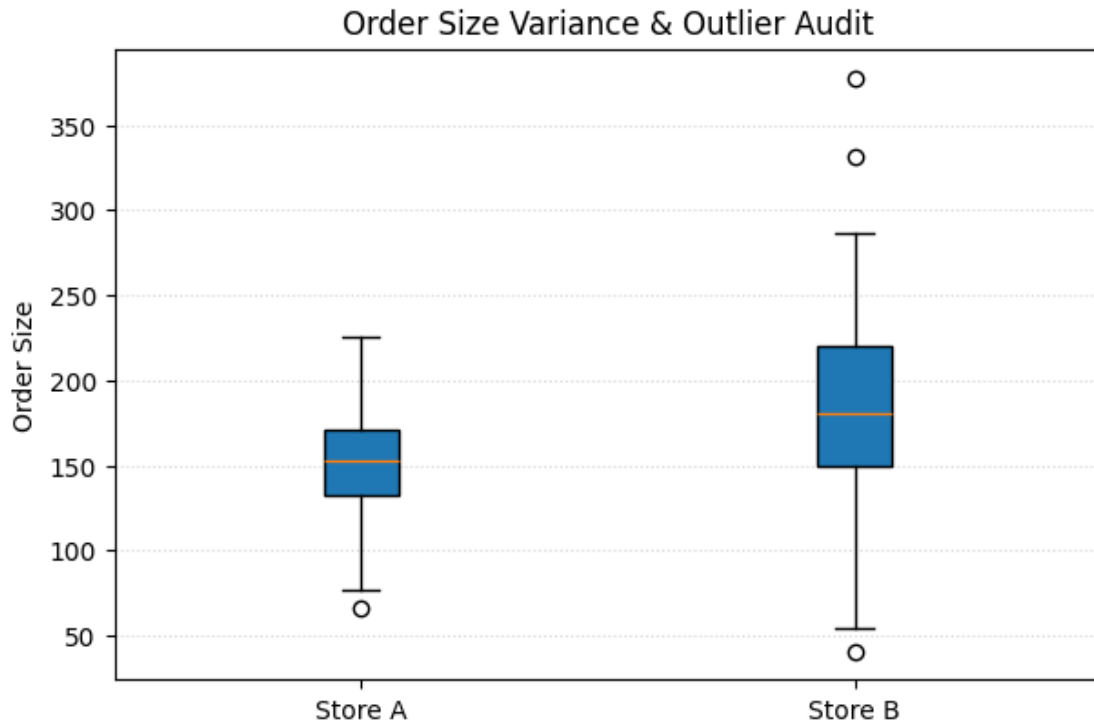
```
[9]: np.random.seed(1)
store_data = pd.DataFrame({
    'Store_A': np.random.normal(150, 30, 300),
    'Store_B': np.random.normal(180, 50, 300)
})

print("--- Creating a Matplotlib Box Plot ---")
plt.figure(figsize=(7, 4.5))
plt.boxplot([store_data['Store_A'], store_data['Store_B']], labels=['Store A', 'Store B'],
            patch_artist=True)
plt.title("Order Size Variance & Outlier Audit")
plt.ylabel("Order Size")
plt.grid(axis='y', linestyle=':', alpha=0.5)
plt.show()
```

--- Creating a Matplotlib Box Plot ---

/tmp/ipykernel_550/2847420940.py:9: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.


```
plt.boxplot([store_data['Store_A'], store_data['Store_B']], labels=['Store A',
'Store B'],
```



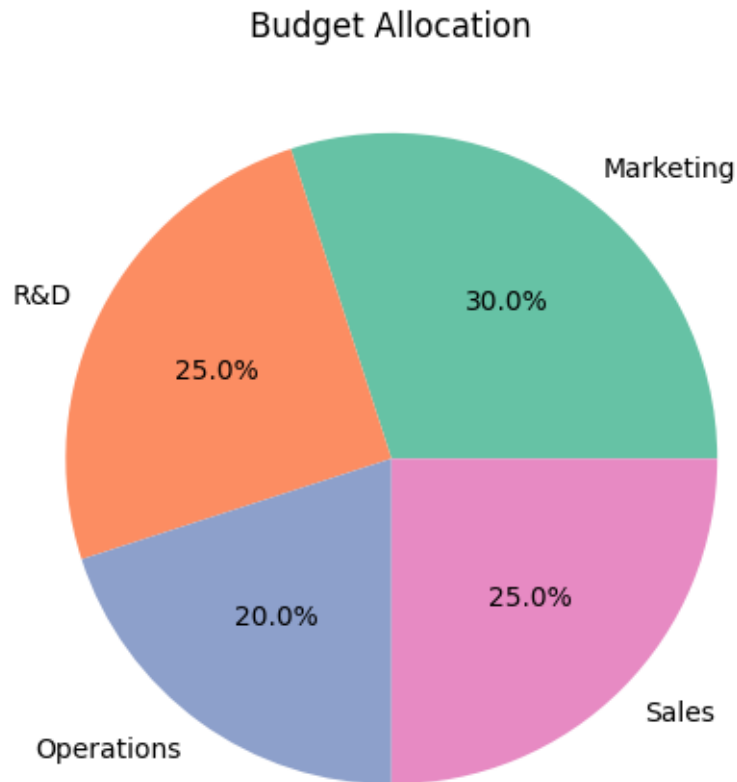
0.1.11 6. Pie Chart

Description: Shows proportions of a whole as slices of a circle. **Use Case:** Displaying budget allocation or market share breakdown. Code snippet:

```
[10]: labels = ["Marketing", "R&D", "Operations", "Sales"]
      budget = [30, 25, 20, 25]

      print("--- Creating a Matplotlib Pie Chart ---")
      plt.figure(figsize=(5.5, 5.5))
      plt.pie(budget, labels=labels, autopct='%1.1f%%',
              colors=['#66c2a5', '#fc8d62', '#8da0cb', '#e78ac3'])
      plt.title("Budget Allocation")
      plt.show()
```

--- Creating a Matplotlib Pie Chart ---



#

CUSTOMIZATION & STYLING

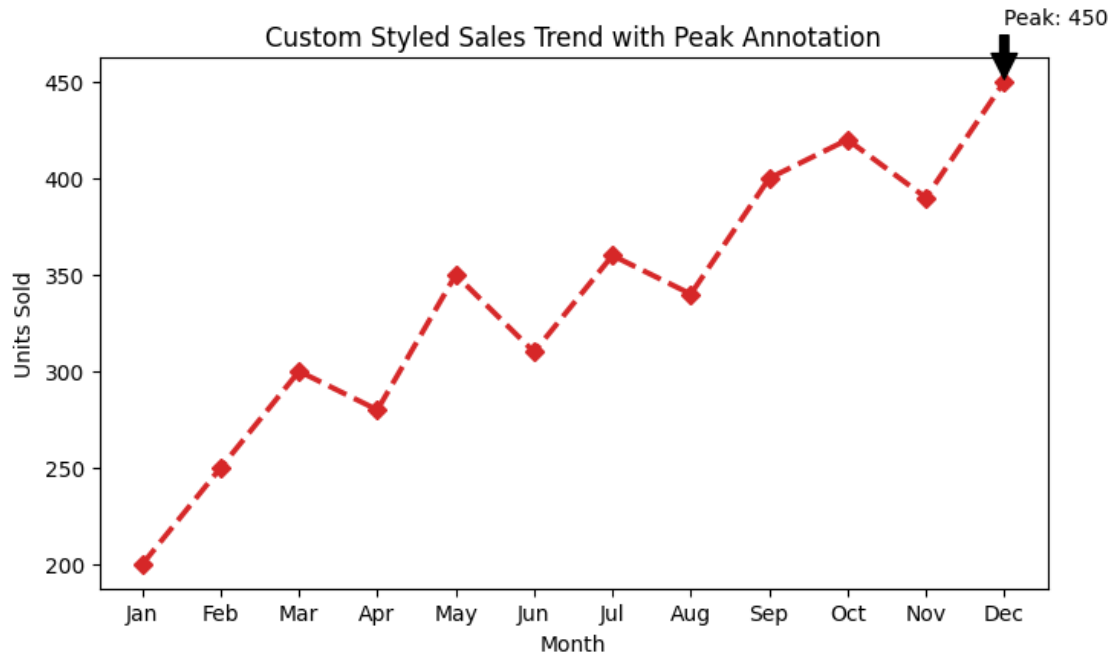
One of Matplotlib's biggest strengths is complete control over every visual element — colors, line styles, annotations, and text — directly through simple keyword arguments. Code snippet demonstrating custom styling and annotations:

```
[11]: print("--- Custom Styled Line Chart with Annotation ---")
plt.figure(figsize=(8, 4.5))
plt.plot(monthly_metrics.index, monthly_metrics['sales_volume'],
         color='#d62728', linewidth=2.5, linestyle='--', marker='D',
         ↪markersize=6)

peak_month = monthly_metrics['sales_volume'].idxmax()
peak_value = monthly_metrics['sales_volume'].max()
plt.annotate(f'Peak: {peak_value}', xy=(peak_month, peak_value),
            xytext=(peak_month, peak_value + 30),
            arrowprops=dict(facecolor='black', shrink=0.05))
```

```
plt.title("Custom Styled Sales Trend with Peak Annotation")
plt.xlabel("Month")
plt.ylabel("Units Sold")
plt.show()
```

--- Custom Styled Line Chart with Annotation ---



#

ADVANTAGES OF MATPLOTLIB

- **Full Control:** Every pixel of a chart — colors, ticks, fonts, layout — can be customized.
- **Wide Chart Support:** From simple 2D line plots to complex 3D visualizations and geographic maps.
- **Foundation Library:** Seaborn, Pandas' `.plot()`, and many other tools are built directly on top of it.
- **Publication Quality:** Extremely fine control makes it ideal for scientific papers and reports.

#

Conclusion

In short, Matplotlib is the foundational visualization tool in Python's data ecosystem. While it requires more code than higher-level libraries to produce a polished chart, it rewards that effort with complete control over every visual element. Whether it's a quick exploratory line plot or a fully-annotated, publication-ready figure, Matplotlib can build it — which is exactly why nearly every other Python plotting library, including Seaborn, is built on top of it.